

Découverte de Pattern Functional Dependencies

Une approche exhaustive face à une approche guidée par LLM

Université Paris Dauphine
Master 2 MIAAGE APP — Data Quality
Enseignant : Khalid Belhajjame

Abdrahamane CAMARA
Yuxuan CHEN
Gide FOMAT
Hongxiang LIN
Liya XU

1. Introduction

Quand on travaille avec des données réelles, on remarque vite que la plupart des contraintes qui régissent ces données ne sont jamais écrites quelque part. Personne n'écrit explicitement que les codes postaux qui commencent par 606 correspondent à Chicago, mais les données le montrent. Ces règles implicites, on les utilise sans s'en rendre compte pour détecter des erreurs, valider des imports, comprendre la structure des données.

Le but de ce projet est d'automatiser la découverte de ces règles. On s'intéresse aux Pattern Functional Dependencies (PFDs), qui généralisent les dépendances fonctionnelles classiques en autorisant des motifs comme les préfixes ou les premiers mots plutôt que de l'égalité stricte sur la valeur entière.

On a exploré le problème sous deux angles :

- Une approche classique par énumération exhaustive : on génère tous les patterns candidats sur chaque colonne, on teste toutes les paires $X \rightarrow Y$ possibles, et on valide chacune statistiquement.
- Une approche guidée par LLM (Workflow 2 du cours) : on demande à un modèle de langage de proposer directement les candidats à tester, en lui montrant le schéma et un échantillon des données.

On a testé ces deux approches sur trois datasets et avec trois variantes de LLM (DeepSeek 1.5B, DeepSeek 8B et Groq Llama 3.1). Le but est de comparer non seulement les performances mais aussi la nature des règles trouvées.

2. Définitions

2.1 Dépendance fonctionnelle classique

Une dépendance fonctionnelle (FD) $X \rightarrow Y$ est satisfaite si, pour tous tuples t et t' de la relation, $t[X] = t'[X]$ implique $t[Y] = t'[Y]$. Deux lignes qui ont la même valeur sur X doivent avoir la même valeur sur Y . Cette définition est trop stricte pour des données réelles qui contiennent presque toujours des erreurs.

2.2 FD approximative

On accepte qu'une fraction ϵ des tuples viole la règle. La confiance d'une FD est le ratio entre les tuples qui la respectent et ceux qui la déclenchent. Une FD est valide si $\text{confiance} \geq (1 - \epsilon)$. Dans nos expériences, $\epsilon = 0,1$, soit une confiance minimale de 90%.

2.3 Pattern Functional Dependency

Les PFDs vont plus loin en remplaçant l'égalité par des motifs. Au lieu de comparer ZIP entier à ZIP entier, on compare $\text{prefix}(\text{ZIP}, 3)$ à CITY. L'exemple canonique du cours est $\text{ZIP startswith '606'} \rightarrow \text{CITY} = \text{'Chicago'}$. Cette généralisation permet de capturer des règles que les FD classiques ne peuvent pas exprimer.

2.4 Métriques

Trois métriques évaluent une PFD candidate :

- Support : nombre de lignes où le motif de X matche. Un support trop faible donne des règles peu fiables.
- Confidence : fraction des tuples du support où Y matche aussi. C'est le critère principal de validité.
- Violations : tuples qui matchent X mais pas Y. Elles désignent souvent des erreurs de données.

3. Approche classique : pipeline d'énumération

L'approche classique implémente le pipeline en 5 étapes décrit dans le cours. Le code est organisé en 4 modules indépendants qui s'enchaînent.

3.1 Pipeline en 5 étapes

Étape 1 : Extraction de patterns

Pour chaque colonne, on extrait les patterns candidats : préfixes de longueur 1 à $k_{\max}$, premiers mots (`first_token`), et valeurs exactes. Les valeurs NaN sont ignorées. Les colonnes numériques sont converties en chaînes pour éviter les artefacts comme '20852.0'.

Étape 2 : Groupement par pattern

Les tuples qui partagent le même pattern sur X forment un groupe. Concrètement c'est un `groupby` pandas. On filtre les groupes en dessous d'un support minimum (5 par défaut) car des groupes trop petits ne sont pas représentatifs.

Étape 3 : Génération des candidats

Pour chaque pattern P sur une colonne X, et pour chaque autre colonne Y, on construit le candidat 'P → valeur majoritaire de Y dans le groupe'. On évite les candidats triviaux où $X = Y$.

Étape 4 : Validation par support et confiance

Pour chaque candidat, on calcule support et confiance. Si support ≥ 5 et confiance ≥ 0.9 ($\epsilon = 0.1$), la PFD est validée.

Étape 5 : Généralisation et élagage

Deux opérations consolident les règles. La fusion regroupe les règles qui partagent (`col_X`, `col_Y`, `val_Y`) et teste si leur plus long préfixe commun reste valide. L'élagage supprime les règles dominées par une règle plus générale. Par exemple, si 'startswith ZIP=6' → 'Chicago' est valide, alors 'startswith ZIP=606' → 'Chicago' est redondante.

3.2 Limitation : explosion combinatoire

L'algorithme teste mécaniquement tous les candidats sans comprendre leur sens. Sur `t2` (entreprises de Chicago), il génère 23 469 candidats et en valide 7 627. Après généralisation, il reste 2 608 règles. Beaucoup sont triviales (tout commence par quelque chose → COUNTRY = United States, car toutes les entreprises sont américaines) ou pointent vers des relations sans intérêt métier (FAX → CITY, PHONE → EMPLOYER_ID).

4. Approche guidée par LLM (Workflow 2)

4.1 Principe

Au lieu d'énumérer mécaniquement, on demande à un LLM de proposer les candidats prometteurs. Le cours (slide 27) décrit deux appels au LLM successifs.

4.2 Premier appel : transformations

Le premier prompt envoie au LLM le schéma du dataset (liste des colonnes) et 5 lignes d'exemple. Il lui demande quelles transformations de patterns appliquer sur quelles colonnes. Le LLM répond avec une liste JSON du type :

```
{"transformations": [{"colonne": "ZIP", "type": "prefix", "longueur": 3}, ...]}
```

4.3 Deuxième appel : candidats

Le deuxième prompt reprend les transformations proposées et demande au LLM quels candidats $X \rightarrow Y$ tester en priorité. La réponse contient une liste de candidats au format JSON, chacun avec col_X, pattern_type, longueur (optionnelle) et col_Y.

4.4 Correction du prompt : contraintes sur les colonnes

Lors des premières expériences, les petits LLMs ont massivement inventé des noms de colonnes inexistants. Sur t1 (employés), Groq a proposé des candidats sur ZIP, PHONE, CITY et NAME alors que ces colonnes n'existent pas dans t1 — il a copié l'exemple générique du prompt au lieu de lire le schéma fourni. Résultat : 15 candidats tous rejetés, 0 PFD trouvée.

On a corrigé le prompt en injectant explicitement la liste exacte des colonnes du dataset et en interdisant d'en inventer. Cette correction a fait passer les résultats de 0 à 107 PFDs sur t1 avec Groq. Ce point est analysé plus en détail dans la section 6.1.

4.5 Validation des candidats

Une fois les candidats reçus, on les valide avec exactement le même module `verifier_pfd` que l'approche classique. C'est important : ça garantit une comparaison équitable. Les candidats qui inventent des colonnes ou qui n'atteignent pas les seuils sont rejetés.

5. Résultats expérimentaux

5.1 Datasets utilisés

Dataset	Description	Lignes	Colonnes
t1	Employés du comté de Montgomery	9 101	9
t2	Employeurs de la ville de Chicago	3 502	13
t3	Licences d'alcool du Maryland	1 077	9

5.2 Configuration des LLMs

Trois variantes de LLM ont été testées :

- DeepSeek-r1:1.5b — modèle local via Ollama, version légère (1 Go), très rapide mais peu robuste sur les instructions complexes.
- DeepSeek-r1:8b — modèle local via Ollama, version plus lourde (5 Go), bien plus précise mais 100x plus lente.
- Groq llama-3.1-8b-instant — API en ligne, gratuite, très rapide (réponses en moins de 2 secondes).

5.3 Tableau de comparaison global

Dataset	Méthode	PFDs	Candidats testés	Temps
t1	Classique	870	~15 000	33 s
t1	Guided + Groq	107	12	2,2 s
t1	Guided + DeepSeek 1.5b	0	3	38 s
t2	Classique	2 608	~23 000	56 s
t2	Guided + Groq	128	15	3,8 s
t2	Guided + DeepSeek 1.5b	213	16	2,5 s
t2	Guided + DeepSeek 8b	227	15	479 s
t3	Classique	271	~5 400	11 s
t3	Guided + Groq	0	2	49 s
t3	Guided + DeepSeek 1.5b	0	3	58 s
t3	Guided + DeepSeek 8b	108	15	495 s

5.4 Observations principales

Le guided search est nettement plus rapide quand il fonctionne. Sur t2, la méthode classique met 56 secondes pour 2 608 PFDs alors que Groq met 3,8 secondes pour 128 PFDs (15 fois plus rapide). DeepSeek 8b prend 479 secondes à cause du modèle plus lourd qui tourne sur CPU.

Le guided trouve beaucoup moins de PFDs mais c'est attendu : il ne teste que 15 candidats au lieu de 15 000 à 23 000. La question importante est : trouve-t-il les bonnes PFDs ?

DeepSeek 1.5b échoue sur deux datasets sur trois (t1 et t3 → 0 PFD). Le petit modèle est incapable de respecter les contraintes du prompt et invente des colonnes ou copie les placeholders. C'est un résultat important : la taille du modèle compte plus que sa rapidité.

Groq fonctionne bien sur t1 et t2 mais échoue complètement sur t3. Le LLM a littéralement copié les placeholders du prompt corrigé (col_X = 'NOM_EXACT'), ce qui montre une faiblesse du modèle 8b de Llama 3.1 sur des prompts complexes.

6. Analyse des résultats

6.1 Problème d'hallucination des colonnes

Le résultat le plus surprenant des premières expériences a été l'échec complet du LLM sur t1. Le dataset t1 contient les colonnes 'Full Name', 'Gender', 'Department', 'Division', 'Position Title'. Groq a proposé des candidats sur ZIP → CITY, PHONE → STATE, NAME → FIRST_NAME — autant de colonnes qui n'existent pas. Tous les candidats ont été rejetés à la validation.

La cause est claire en relisant le prompt : on lui montrait un exemple générique avec ZIP/CITY/STATE pour illustrer le format JSON attendu. Le LLM a interprété ces noms comme des recommandations plutôt que comme des placeholders. Il a copié l'exemple littéralement.

La correction a consisté à injecter explicitement la liste des colonnes réelles dans le prompt et à ajouter une consigne stricte : 'Tu dois utiliser UNIQUEMENT les colonnes qui existent vraiment dans ce dataset. Les colonnes disponibles sont EXACTEMENT : [...]. N'invente AUCUNE autre colonne.' Après correction, t1 + Groq est passé de 0 à 107 PFDs valides.

Cette observation est importante pour la rédaction de prompts pour LLM : les exemples génériques sont des sources d'erreurs et il faut toujours contraindre explicitement la sortie au schéma réel des données.

6.2 Effet de la taille du modèle

La comparaison entre DeepSeek 1.5b et 8b sur t2 illustre l'effet de la taille du modèle :

- DeepSeek 1.5b : 213 PFDs en 2,5 secondes mais avec des candidats parfois mal formés (pattern_type 'longueur' inexistant).
- DeepSeek 8b : 227 PFDs en 479 secondes, candidats mieux structurés et plus pertinents.

L'écart de qualité est plus visible sur les datasets difficiles. Sur t3, le 1.5b échoue (0 PFD) alors que le 8b trouve 108 PFDs cohérentes (Zip → State='MD'). Le petit modèle ne fait que prédire des suites de tokens probables sans vraiment comprendre la structure du prompt ; le gros modèle suit mieux les instructions.

Le compromis est évident : le 8b est qualitativement supérieur mais 200 fois plus lent. Pour un usage industriel, Groq via API offre un meilleur rapport qualité/vitesse quand on a une connexion réseau fiable.

6.3 Comparaison qualitative des règles

Les chiffres bruts ne disent pas tout. La nature des règles trouvées est plus parlante. Voici trois exemples de règles découvertes par le guided search sur t1 (employés) :

Department['POL'] → Department Name='Department of Police' (conf 100%, sup 1835)

Department['HHS'] → Department Name='Department of Health and Human Services' (conf 100%, sup 1512)

Division['MSB'] → Assignment Category='Fulltime-Regular' (conf 100%, sup 287)

Ces règles sont des bijections code-libellé que tout référentiel métier devrait respecter. Elles sont sémantiquement riches et utilisables pour valider des saisies futures.

À l'inverse, voici trois règles trouvées par la méthode classique sur t2 :

FAX → CITY

PHONE → EMPLOYER_ID

ADDRESS_2 → CREATED_DATE

Ces règles n'ont aucun sens. Elles existent statistiquement (le calcul de confiance les valide) mais elles ne capturent aucune connaissance métier. La méthode classique les trouve mécaniquement parce qu'elle teste tous les candidats.

6.4 Recouvrement entre les approches

Sur t2, la méthode classique trouve 2 608 PFDs, le guided Groq en trouve 128, et seulement 15 paires (col_X, col_Y) sont communes aux deux. Cela signifie que les deux approches explorent en partie des espaces différents :

- La classique trouve des PFDs sur des combinaisons de colonnes que le LLM ignore (souvent du bruit, mais parfois des cas légitimes que le LLM n'a pas vus).
- Le guided trouve quelques PFDs sémantiquement pertinentes (ZIP["606"] → CITY="Chicago") qui sont bien dans les résultats classiques, mais avec une priorité d'affichage.

Cela suggère qu'une approche hybride serait optimale : utiliser le guided search pour identifier rapidement les PFDs sémantiquement riches, et l'énumération classique pour la couverture exhaustive.

7. Les PFDs comme outil de diagnostic de qualité

Au-delà de la découverte de règles métier, les PFDs sont aussi un outil de diagnostic des problèmes de qualité des données. Les violations d'une PFD presque vraie pointent souvent vers des erreurs réelles.

7.1 Incohérences de casse dans t2

La colonne CITY de t2 contient Chicago écrit de plusieurs façons : 'Chicago' (2116 fois), 'chicago' (16), 'CHICAGO' (9), 'CHicago' (3). La règle ZIP startswith '606' → CITY='Chicago' a quelques violations qui sont en fait des Chicago mal orthographiés. La PFD signale donc indirectement un problème de standardisation.

7.2 Valeurs aberrantes dans STATE

La colonne STATE de t2 contient principalement des codes US à 2 lettres, mais aussi quelques valeurs qui n'en sont pas : FR (6 fois), QE (3), BC (1). Les violations de la règle ZIP → STATE permettent d'identifier ces enregistrements probablement étrangers ou mal saisis.

7.3 Formats hétérogènes dans PHONE

La colonne PHONE de t2 mélange plusieurs formats : '312-920-1500' (3053 fois), '(312) 920-1500' (123), '312.920.1500' (49). Le guided search trouve la règle startswith(PHONE, '312') → CITY='Chicago', mais l'énumération exhaustive trouve plutôt startswith(PHONE, '(312)') avec un support de seulement 46. Les deux versions coexistent à cause du format non standardisé — c'est un signal clair pour une étape de nettoyage.

8. Discussion

8.1 Couverture vs pertinence

Le résultat principal de notre étude est qu'il n'y a pas d'approche meilleure dans l'absolu. Les deux approches incarnent des choix méthodologiques différents.

L'énumération classique offre une couverture massive (2 608 PFDs sur t2) mais avec un signal-sur-bruit faible : beaucoup de règles sont triviales ou sans intérêt. Le filtrage manuel est nécessaire.

Le guided search offre une couverture limitée (128 PFDs sur t2 avec Groq) mais avec un signal-sur-bruit élevé : les règles proposées sont presque toutes sémantiquement pertinentes (bijections code-libellé, préfixes ZIP géographiques).

Le choix dépend du cas d'usage. Pour un audit complet lors d'une migration de schéma, l'énumération classique est mieux adaptée. Pour générer rapidement des règles de validation immédiatement utilisables, le guided search suffit.

8.2 Coût et fiabilité des LLMs

Trois facteurs limitent l'utilisation pratique des LLMs :

- La stochasticité : deux runs successifs peuvent produire des candidats différents. C'est gérable en fixant la température à 0,1 mais pas éliminable.
- La dépendance au schéma fourni : sans contraintes explicites dans le prompt, les LLMs inventent des colonnes. C'est un point essentiel à anticiper.
- Le coût : DeepSeek 8b prend 8 minutes par dataset sur CPU. Groq est rapide mais soumis à des quotas. Les coûts financiers, latence et quotas sont à comparer aux ressources CPU/RAM de la méthode classique.

8.3 Limites de notre étude

Notre étude se base sur un seul run par configuration. La stochasticité des LLMs n'est donc pas quantifiée. Idéalement il faudrait répéter chaque expérience 5 à 10 fois pour mesurer la variance.

Nous avons aussi limité `k_max` à 3 pour les préfixes. Un `k_max` plus grand permettrait de capturer des codes métier longs comme 'BBWLHR' dans `t3`, mais ferait exploser le nombre de candidats. Un compromis serait d'augmenter `k_max` uniquement pour les colonnes que le LLM identifie comme contenant des codes structurés.

Enfin, le Workflow 3 (Agent in the Loop) n'a pas été implémenté faute de temps. Il consisterait à faire une boucle où l'algorithme valide les candidats du LLM, lui renvoie les violations pour qu'il raffine ses propositions, et itère jusqu'à convergence. C'est une piste pour une étude ultérieure.

9. Conclusion

Ce projet a comparé deux approches de découverte de Pattern Functional Dependencies : une approche classique par énumération exhaustive et une approche guidée par LLM. Trois datasets ont été utilisés (t1 employés, t2 entreprises, t3 licences) et trois variantes de LLM ont été comparées (DeepSeek 1.5b, DeepSeek 8b, Groq Llama 3.1).

Les conclusions principales sont :

- L'approche classique trouve beaucoup de PFDs (2 608 sur t2) mais avec un fort bruit : beaucoup de règles sont triviales ou sans intérêt métier.
- L'approche guidée trouve moins de PFDs (128 sur t2 avec Groq) mais elles sont sémantiquement plus pertinentes : bijections code-libellé, préfixes ZIP géographiques.
- La taille du modèle compte plus que sa vitesse : DeepSeek 1.5b échoue sur les datasets avec colonnes non standards, alors que le 8b et Groq réussissent.
- La rédaction du prompt est cruciale : sans contraintes explicites sur les colonnes, les LLMs hallucinent et copient les placeholders, ce qui fait échouer complètement la découverte.
- Les violations de PFDs presque valides sont elles-mêmes informatives : elles révèlent des incohérences de casse, des formats mélangés, des erreurs de saisie. La découverte de PFDs est donc autant un outil de diagnostic de qualité qu'un outil de découverte de contraintes.

Une suite naturelle de ce travail serait d'implémenter le Workflow 3 (Agent in the Loop) où le LLM raffine itérativement ses propositions en fonction des violations observées, et de tester une approche hybride combinant l'énumération exhaustive pour la couverture et le LLM pour le filtrage sémantique.